



Universitat d'Alacant
Universidad de Alicante



GRADO EN INGENIERÍA INFORMÁTICA
CURSO ACADÉMICO 2025-2026

ANÁLISIS Y DISEÑO DE ALGORITMOS

GRUPO 8 - PRÁCTICA FINAL



JULIÁN HINOJOSA GIL - DNI 48795869-N

Índice

1. Estructuras de datos	1
1.1. Nodo	1
1.2. Lista de nodos vivos	1
2. Mecanismos de poda	2
2.1. Poda de nodos no factibles	2
2.2. Poda de nodos no prometedores	2
3. Cotas pesimistas y optimistas	3
3.1. Cota pesimista inicial (inicialización)	3
3.2. Cota pesimista del resto de nodos	4
3.3. Cota optimista	4
4. Otros medios empleados para acelerar la búsqueda	5
4.1. Optimización espacial: Reconstrucción inversa del camino óptimo	5
4.2. Parámetros y directivas de optimización del compilador	6
5. Estudio comparativo de distintas estrategias de búsqueda	6
5.1. Discusión y análisis de resultados	7
5.1.1. El colapso asintótico de la estrategia LIFO	7
5.1.2. Análisis de la paradoja FIFO frente a Best-First	7
5.2. Justificación de la estrategia seleccionada	8
6. Tiempos de ejecución	8
6.1. Entorno de pruebas y especificaciones de hardware	8
6.2. Resultados experimentales	8
Bibliografía	10

1. Estructuras de datos

1.1. Nodo

Para la representación de los estados dentro del espacio de búsqueda, se ha diseñado una estructura compacta denominada `Nodo`. Cada instancia almacena exclusivamente los tipos primitivos mínimos necesarios para guiar la exploración, evitando la sobrecarga en el espacio de estados.

Las variables miembro que componen la estructura y sus cometidos son:

- `i, j`: Enteros que indexan la posición bidimensional actual del explorador dentro de la matriz del laberinto.
- `coste_acumulado`: Variable de tipo `long` que contabiliza el número real de casillas transitadas desde el origen $(0, 0)$ hasta la posición actual (equivalente al coste real $g(n)$).
- `cota_optimista`: Registro de tipo `long` que almacena la suma del coste acumulado y el valor devuelto por la función heurística de estimación hasta la meta. Representa la longitud ideal del camino completo si no se encontraran muros adicionales.

Como optimización crítica de ingeniería de software, se ha omitido la inclusión de cualquier contenedor dinámico (como `std::vector`) dentro del propio nodo. Esto previene la asignación continua en el *heap* y la copia de memoria intermedia durante la ramificación. Asimismo, se incluye la sobrecarga del operador de comparación `operator>` con el fin de invertir el comportamiento por defecto de la cola de prioridad.

```

1 struct Nodo {
2     int i, j;                // Posición actual en el laberinto
3     long coste_acumulado;    // Pasos reales dados desde el origen
4     long cota_optimista;     // Coste acumulado + estimación heurística
5
6     // Sobrecarga del operador mayor para implementar un Min-Heap
7     bool operator>(const Nodo& otro) const {
8         return cota_optimista > otro.cota_optimista;
9     }
10 };

```

Listing 1: Definición de la estructura `Nodo`

1.2. Lista de nodos vivos

La Lista de Nodos Vivos (LNV) gestiona los frentes de onda generados y pendientes de expansión. Se ha seleccionado como contenedor principal una cola de prioridad estructurada como un montículo de mínimos (*Min-Heap*), parametrizada con la relación de orden interna definida en el nodo mediante `std::greater`.

El criterio de extracción implementado selecciona en tiempo $O(\log N)$ el nodo vivo con la menor `cota_optimista` global (estrategia *Best-First* o *Least-Cost Search*). Esto garantiza que el algoritmo guíe la exploración de forma informada hacia el objetivo físico, minimizando el desarrollo de ramas secundarias divergentes.

Siguiendo el temario teórico, se evaluaron otras dos estructuras de almacenamiento para la LNV que finalmente se descartaron para la versión de producción:

- `std::queue` (Estrategia FIFO): Implementa una búsqueda en anchura ciega. Extrae los elementos en orden estricto de llegada, lo que provoca una expansión radial simétrica (en forma de mancha de aceite) que satura la memoria con millones de nodos no óptimos al carecer de dirección heurística.
- `std::stack` (Estrategia LIFO): Implementa una búsqueda en profundidad. Explora caminos de forma ciega hasta alcanzar un elemento no factible o una hoja, generando una altísima inestabilidad y picos de computación masivos si se interna prematuramente en zonas densas de muros lejanas a la meta.

```

1 // Lista de Nodos Vivos basada en estrategia guiada por cota optimista
2 std::priority_queue<Nodo, std::vector<Nodo>, std::greater<Nodo>> LNV;

```

Listing 2: Declaración de la Lista de Nodos Vivos final

2. Mecanismos de poda

2.1. Poda de nodos no factibles

La poda por falta de factibilidad constituye el primer filtro de confinamiento del espacio de estados. Se aplica de forma inmediata durante la generación de los ocho vecinos colindantes a la posición actual, impidiendo que se incorporen a la Lista de Nodos Vivos (*LNV*) aquellos estados que violan las restricciones físicas o estructurales del problema.

Un nodo hijo es descartado automáticamente y contabilizado bajo el registro `st.descartados_no_factibles` si incurre en alguna de las siguientes condiciones de contorno:

- **Desbordamiento de índices:** Si las coordenadas calculadas (*isig*, *jsig*) se sitúan fuera de los límites dimensionales indexados de la matriz ($n \times m$), evitando accesos ilegales a memoria.
- **Colisión con obstáculo:** Si la casilla destino contiene el valor binario 0, el cual simboliza un muro o sector infranqueable dentro del diseño del laberinto.

```

1 // Verificación de límites de la matriz y transitabilidad de la celda
2 if (isig >= 0 && isig < n && jsig >= 0 && jsig < m && maze[isig][jsig] == 1) {
3     // El nodo es factible, se procede a evaluar su viabilidad económica
4 } else {
5     // Cortado por chocar con un muro o salirse del mapa
6     st.descartados_no_factibles++;
7 }

```

Listing 3: Control de factibilidad geométrica y estructural

2.2. Poda de nodos no prometedores

Una vez garantizada la factibilidad geométrica de un movimiento, el algoritmo aplica un doble mecanismo de poda por promesa (tanto en fase temprana como tardía) con el objetivo de evitar la exploración de caminos subóptimos y detener el crecimiento exponencial del árbol de derivación.

El descarte de nodos no prometedores se divide operacionalmente en dos estrategias:

1. **Poda Temprana (Filtro previo a la inserción):** Ocurre antes de introducir un nodo hijo en la *LNV*. Se subdivide en dos criterios concurrentes:

- **Por Heurística Futura (Cota Optimista):** Se calcula la cota optimista teórica del hijo (*nueva_cota*). Si este valor es mayor o igual que el récord global actual (*bestSol*), se poda inmediatamente (`st.descartados_no_prometedores++`), ya que matemáticamente es imposible que dicho camino mejore la solución óptima en curso.
- **Por Historial Pasado (Cercado de Ciclos):** Si la cota optimista es válida, se contrasta el *nuevo_coste* acumulado frente al registro histórico estático almacenado en la matriz de control `costs[isig][jsig]`. Si el coste es mayor o igual que un camino previo que ya pisó esa misma celda, el nodo se descarta al representar una ruta redundante o ineficiente.

2. **Poda Tardía (Filtro posterior a la extracción):** Se ejecuta en el instante en que un nodo es extraído de la cima de la cola de prioridad (`LNV.top()`). Dado que los nodos pueden permanecer almacenados en el montículo mientras otras ramas descubren soluciones óptimas, la cota pesimista global (`bestSol`) puede haber disminuido drásticamente en el intervalo de espera. Comprobar la condición justo antes de la expansión de vecinos evita malgastar ciclos de CPU en estados obsoletos.

```

1 // --- A) PODA TARDÍA (Evaluación tras extracción de la LNV) ---
2 if (actual.cota_optimista >= bestSol) {
3     st.prometedores_descartados++;
4     continue; // Descarte del nodo y salto al siguiente ciclo del while
5 }
6
7 // ... (Dentro del bucle de expansión de vecinos tras verificar factibilidad) ...
8
9 // --- B) PODA TEMPRANA (Evaluación heurística futura y matriz de costes) ---
10 if (nueva_cota < bestSol) {
11
12     if (nuevo_coste < costs[isig][jsig]) {
13         costs[isig][jsig] = nuevo_coste;
14         from[isig][jsig] = direccion;
15
16         // Inserción segura del nodo prometedor
17         st.explorados++;
18         LNV.push(hijo);
19     } else {
20         // Cortado por historial redundante en la matriz de costes
21         st.descartados_no_prometedores++;
22     }
23 } else {
24     // Cortado porque su estimación ideal no puede superar al récord actual
25     st.descartados_no_prometedores++;
26 }

```

Listing 4: Implementación de la poda tardía y temprana por cotas

3. Cotas pesimistas y optimistas

El rendimiento de un algoritmo de Ramificación y Poda depende de la calidad y del ajuste de sus funciones de acotación. Las cotas optimistas (teóricas) permiten descartar ramas de forma segura, mientras que las cotas pesimistas (soluciones reales factibles) actúan como el umbral de corte del algoritmo.

3.1. Cota pesimista inicial (inicialización)

Arrancar el bucle principal de exploración con una cota pesimista infinita penaliza drásticamente el rendimiento, obligando al algoritmo a expandir a ciegas las primeras ramas hasta dar casualmente con una hoja. Para evitar este comportamiento, se ha implementado una función auxiliar llamada `greedy_initial_sol` que se ejecuta de forma previa al proceso de ramificación.

Este algoritmo secundario realiza una búsqueda voraz (**Greedy Search**) de coste sub-milisegundo. En cada iteración, evalúa las posiciones vecinas factibles y avanza de manera irrevocable hacia la celda colindante que minimice la distancia heurística al objetivo, controlando las celdas ya pisadas mediante una matriz local de visitados para impedir bucles infinitos.

Si el algoritmo voraz logra alcanzar la meta $(n-1, m-1)$, devuelve la longitud exacta del camino hallado. Este valor se asigna inmediatamente a la variable global `bestSol`, fijando un récord inicial ajustado desde el

milisegundo cero. Gracias a este umbral inicial, la cola de prioridad puede aplicar la poda tardía y temprana con gran severidad desde las primeras iteraciones del bloque principal.

```

1 // Cálculo de la solución subóptima inicial en caliente
2 long greedy_sol = greedy_initial_sol(maze, best_path);
3 if (greedy_sol != INFINITO) {
4     bestSol = greedy_sol; // Umbral de poda inicializado de forma compacta
5 }

```

Listing 5: Invocación e inicialización de la cota pesimista inicial

3.2. Cota pesimista del resto de nodos

A lo largo de la ejecución del bucle general de ramificación, la cota pesimista global (bestSol) se encarga de almacenar la longitud del camino óptimo provisional descubierto hasta el momento.

Evaluando el compromiso entre la eficacia de la poda y la complejidad temporal de los cálculos, se determinó omitir la estimación de cotas pesimistas dinámicas intermedias en cada nodo hijo (operación que requeriría lanzar búsquedas voraces iterativas en cada expansión de estados, ralentizando la CPU). En su lugar, el algoritmo actualiza la cota pesimista únicamente de forma pasiva cuando un nodo calificado como hoja (nodo completado que ha alcanzado físicamente las coordenadas de destino) se extrae del frente de la LNV.

Si el parámetro `coste_acumulado` del nodo hoja es estrictamente inferior al valor vigente en `bestSol`, se sobrescribe el récord global y se incrementa el contador estadístico `st.actualizaciones_desde_hoja`.

```

1 // --- B) HEMOS LLEGADO A LA META? (Nodo Hoja) ---
2 if (actual.i == n - 1 && actual.j == m - 1) {
3     st.hojas++;
4     if (actual.coste_acumulado < bestSol) {
5         bestSol = actual.coste_acumulado; // Reducción del umbral de poda global
6         st.actualizaciones_desde_hoja++;
7
8         // Reconstrucción del camino óptimo a través del historial de movimientos
9         // ...
10    }
11    continue;
12 }

```

Listing 6: Actualización de la cota pesimista global en nodos hoja

3.3. Cota optimista

La función de cota optimista estima conceptualmente el escenario ideal de pasos pendientes desde la celda actual hasta el destino, asumiendo la hipótesis de que no se interpondrá ningún muro intermedio. Para que el algoritmo garantice la convergencia hacia la solución óptima global, es un requisito matemático estricto que esta función sea **admisible**, es decir, que subestime o iguale el coste real del camino, pero nunca lo sobrestime.

Dado que el enunciado permite desplazamientos libres en ocho direcciones (incluyendo movimientos diagonales con un coste unitario idéntico a las marchas ortogonales), la función de cota optimista idónea es la ****Distancia de Chebyshev****. Esta métrica calcula el valor máximo entre la diferencia absoluta de filas y columnas pendientes hasta la esquina inferior derecha del mapa:

$$\text{Chebyshev}(i, j) = \max((n - 1) - i, (m - 1) - j)$$

Es una cota admisible pura: en un tablero ideal libre de obstáculos, el número de pasos mínimos necesarios para conectar dos puntos considerando movimientos diagonales es exactamente el máximo de sus componentes de distancia. Al no considerar la existencia de muros, la función cumple la condición de subestimación necesaria para guiar la cola de prioridad de forma óptima.

```

1 // Función de estimación heurística: Distancia de Chebyshev al destino
2 long estimacion(int filas, int columnas, int i, int j) {
3     return std::max(filas - 1 - i, columnas - 1 - j);
4 }

```

Listing 7: Implementación analítica de la cota optimista

4. Otros medios empleados para acelerar la búsqueda

Además de los mecanismos estándar de acotación y poda por cotas, el rendimiento final del programa se ha optimizado actuando sobre dos cuellos de botella críticos: la sobrecarga en la gestión de memoria dinámica (*heap allocation overhead*) y la optimización del código máquina a nivel de compilador.

4.1. Optimización espacial: Reconstrucción inversa del camino óptimo

Una aproximación inicial ingenua para recuperar la secuencia de movimientos ganadora consiste en almacenar un contenedor local (como un `std::vector<unsigned>`) dentro de cada instancia de la estructura `Nodo`. Sin embargo, esta estrategia introduce una penalización inasumible: cada vez que el algoritmo ramifica y añade un hijo a la *LNV*, la CPU se ve obligada a duplicar el vector del nodo padre en el *heap*, provocando miles de llamadas al sistema (`malloc/free`), fragmentación de memoria y fallos de caché (*cache misses*) que degradan el rendimiento exponencialmente.

Para solventar este problema, se diseñó un mecanismo de **reconstrucción inversa asintóticamente óptimo** en espacio y tiempo:

- Se reserva una única matriz estática bidimensional de control denominada `from` de dimensiones $n \times m$, indexada con enteros sin signo.
- Cada vez que un movimiento es validado por las funciones de poda y se inserta en la cola, se anota en `from[isig][jsig]` el identificador numérico de la dirección empleada para pisar dicha celda. Esto requiere un coste espacial despreciable de $O(n \times m)$ frente al coste dinámico de los vectores en los nodos.
- Únicamente cuando un nodo hoja procesado en la meta mejora la cota pesimista global (`bestSol`), se ejecuta un bucle `while` que realiza un recorrido inverso saltando de celda en celda desde las coordenadas de destino $(n - 1, m - 1)$ hacia el origen $(0, 0)$ guiado por el historial de la matriz `from`. El coste temporal de esta reconstrucción es estrictamente lineal $O(L)$, donde L es la longitud de la ruta actual, y se ejecuta un número mínimo de veces a lo largo del programa (solo cuando hay actualizaciones de `record`).

```

1 // Reconstrucción asintótica de la ruta óptima mediante la matriz de procedencia
2 best_path.clear();
3 int curr_i = n - 1, curr_j = m - 1;
4 while (curr_i != 0 || curr_j != 0) {
5     unsigned move = from[curr_i][curr_j];
6     best_path.push_back(move);
7
8     // Desplazamiento inverso restando los incrementos de dirección
9     curr_i -= dirs[move][0];
10    curr_j -= dirs[move][1];
11 }
12 // Inversión final del vector para recuperar el orden cronológico del camino
13 std::reverse(best_path.begin(), best_path.end());

```

Listing 8: Algoritmo de reconstrucción inversa en el nodo hoja

4.2. Parámetros y directivas de optimización del compilador

Siguiendo las recomendaciones del enunciado de la asignatura, el entorno de compilación automatizado a través del archivo `makefile` se ha configurado para extraer el máximo rendimiento del hardware del laboratorio.

Se ha prescindido por completo de la directiva de depuración `-g` (la cual incrusta símbolos de depuración ralentizando los saltos de línea de ejecución) y se ha introducido la bandera de optimización agresiva `-O3`. Esta directiva habilita al compilador `g++` para aplicar optimizaciones avanzadas sobre nuestro código máquina, tales como el desenrollado automático de bucles (***loop unrolling***) en el for de las 8 direcciones, la vectorización de instrucciones SIMD, la propagación de constantes y la expansión en línea (***inlining***) de funciones de cálculo intensivo y repetitivo como `estimacion()`. El impacto de esta medida reduce los tiempos netos de respuesta de la CPU a una tercera parte de la marca original.

5. Estudio comparativo de distintas estrategias de búsqueda

Para evaluar empíricamente la influencia de la estructura de la Lista de Nodos Vivos (***LNV***) sobre la eficiencia espacial y temporal del algoritmo, se ha sometido el archivo de test `500-bb.maze` a tres configuraciones distintas, modificando exclusivamente el criterio de extracción de los estados vivos: estrategia guiada por cota (***Best-First*** mediante `std::priority_queue`), búsqueda en anchura ciega (FIFO mediante `std::queue`) y búsqueda en profundidad (LIFO mediante `std::stack`).

El Cuadro 1 sintetiza los valores estadísticos agregados y los tiempos medios computados tras aplicar la metodología experimental de descarte de la primera ejecución por caché en frío.

Cuadro 1: Métricas detalladas por estrategia de exploración (Fichero: 500-bb.maze)

Métrica Estadística	Best-First (Priority Queue)	Anchura (FIFO)	Profundidad (LIFO)
Longitud Solución (Pasos)	5.303	5.303	5.303
Nodos Visitados (N_{visit})	287.313	277.553	1.204.657.273
Nodos Explorados ($n_{explored}$)	35.974	34.715	150.639.938
Nodos Hoja (N_{leaf})	1	1	8.245
Descartados No Factibles	37.435	35.972	156.791.645
Descartados No Prometedores	213.904	206.866	897.225.690
Prometedores Descartados	59	20	49.534
Actualizaciones desde Hoja	1	1	8.245
Tiempo Medio de CPU	4,63 ms	2,81 ms	6.890,88 ms

5.1. Discusión y análisis de resultados

5.1.1. El colapso asintótico de la estrategia LIFO

Los resultados de la búsqueda en profundidad pura (LIFO) revelan una ineficiencia masiva, elevando el tiempo de respuesta hasta los **6.890,88 ms** y disparando el número de nodos explorados por encima de los **150 millones**.

Al extraer los elementos según el orden de inserción estricto de la pila, el algoritmo carece de una perspectiva global de la distancia hacia la meta. LIFO se interna ciegamente en ramas profundas alejadas geométricamente del objetivo físico. Aunque los mecanismos de poda por cota y matriz de costes históricos se encuentran activos, la naturaleza desordenada de la pila respecto a la cota optimista neutraliza la efectividad de la poda tardía en fases tempranas. Esto obliga al algoritmo a registrar y evaluar **8.245 soluciones hoja** subóptimas antes de converger, multiplicando exponencialmente las ramificaciones inútiles.

5.1.2. Análisis de la paradoja FIFO frente a Best-First

Bajo las leyes teóricas estándar de la Ramificación y Poda, la cola de prioridad (**Best-First**) debería acotar el espacio de estados de forma estrictamente superior a la búsqueda no informada en anchura (FIFO). Sin embargo, los datos empíricos del laberinto 500-bb.maze muestran un comportamiento inverso: la estrategia FIFO explora ligeramente menos nodos (34.715 frente a 35.974) y reduce el tiempo de proceso a casi la mitad (2,81 ms frente a 4,63 ms).

Este fenómeno se fundamenta en la interacción de dos factores técnicos concurrentes:

- Eficacia del Algoritmo Voraz Inicial:** La función de inicialización pesimista `greedy_initial_sol` computa de forma inmediata una solución factible de 5.303 pasos. En esta instancia concreta del laberinto, dicha solución resulta coincidir exactamente con el ****óptimo global absoluto**** del problema. Al arrancar el bucle con un récord insuperable en `bestSol`, el espacio de estados potencial queda drásticamente recortado para ambas estrategias desde la primera iteración.
- Sobrecarga algorítmica de las estructuras de la STL:** Al estar el contorno de búsqueda fuertemente delimitado por la cota voraz, la estrategia en anchura expande los niveles de forma uniforme y simétrica. Aquí entra en juego la complejidad interna de las estructuras de datos: la cola FIFO (`std::queue`) opera con un coste constante de inserción y extracción $O(1)$, mientras que la cola de prioridad (`std::priority_queue`) requiere un coste de reordenación del montículo (**Heapify**) de tipo $O(\log N)$ por cada operación. El coste computacional de mantener las celdas ordenadas por heurística genera una fricción innecesaria en la CPU cuando el espacio ya está acotado, lo que explica la ventaja temporal de FIFO.

5.2. Justificación de la estrategia seleccionada

A pesar del rendimiento puntual favorable de la cola FIFO en este mapa específico, ****se ratifica la estructura de Priority Queue (Best-First) como la arquitectura definitiva de producción**** para el proyecto.

En instancias asimétricas complejas o mapas de tipo K (donde el algoritmo voraz inicial puede encallarse en callejones sin salida devolviendo cotas pesimistas infinitas o muy alejadas de la realidad), la búsqueda ciega en anchura colapsaría de inmediato al expandir millones de nodos en anillos concéntricos concéntricos. La cola de prioridad ordenada por distancia de Chebyshev garantiza una convergencia directa y geométrica hacia el objetivo bajo cualquier escenario crítico, consolidándose como la solución de ingeniería más robusta y consistente.

6. Tiempos de ejecución

6.1. Entorno de pruebas y especificaciones de hardware

Para garantizar la validez y reproducibilidad de los resultados obtenidos, todas las mediciones de rendimiento global se han realizado bajo un entorno controlado en los laboratorios de la Escuela Politécnica IV. Las especificaciones técnicas del hardware institucional empleado son las siguientes:

- **Procesador (CPU):** Intel Core i5-13500 (13.^a generación).
- **Memoria RAM:** 16 GB DDR4.
- **Sistema Operativo:** Ubuntu 24.04.
- **Compilador:** GNU GCC (g++) utilizando la directiva de optimización agresiva -O3 y el estándar C++11.

6.2. Resultados experimentales

Siguiendo la metodología descrita, se han completado 6 ejecuciones por cada archivo de prueba, descartando de forma sistemática el primer lanzamiento para neutralizar la latencia por lectura en frío de disco (Cold Cache). Los datos expuestos en el Cuadro 2 representan la media aritmética estricta de las 5 muestras restantes en caliente.

El tribunal de la asignatura fija un umbral crítico de tolerancia de 15.000 milisegundos (15 segundos) por instancia, penalizando cualquier desbordamiento temporal.

Cuadro 2: Tiempos de proceso medios en la plataforma de hardware institucional

Fichero de test	Tiempo Medio (ms)
100-bb.maze	1,19
200-bb.maze	1,62
300-bb.maze	10,23
500-bb.maze	4,64
700-bb.maze	22,02
900-bb.maze	37,71
k01-bb.maze	25,98
k02-bb.maze	78,55
k03-bb.maze	155,82
k05-bb.maze	445,64
k10-bb.maze	1.501,21

Como se puede observar de forma directa en los resultados recopilados, el resolutor desarrollado cumple holgadamente con los requisitos de eficiencia de la práctica. Incluso ante el escenario de máxima complejidad y densidad de caminos ciegos (k10-bb.maze), el algoritmo converge firmemente en poco más de

un segundo y medio (1.501,21 ms), situándose un orden de magnitud por debajo del límite de exclusión del tribunal.

Bibliografía

1. Morales García, J., Sánchez Cartagena, V. M., y Verdú Mas, J. L. (2026). ***Tema 7: Ramificación y Poda***. Departamento de Lenguajes y Sistemas Informáticos, Universitat d'Alacant.
2. Departamento de Lenguajes y Sistemas Informáticos (2026). ***Enunciado de la Práctica Final: Camino de coste mínimo y Ramificación y Poda***. Grado en Ingeniería Informática, Universitat d'Alacant.